ARQUITECTURA DE SOFTWARE

Taller del 2do corte · Docker

Informe de práctica

Docker con biblioteca-api · Parte 2

Estudiante

Juan Jose Barreto

Sistema operativo

Windows

Octubre de 2026

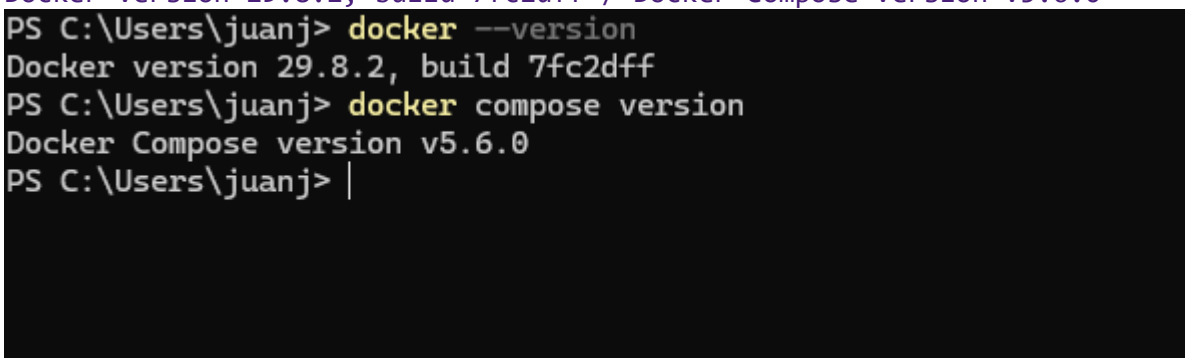
Fase 1: Instalación de Docker

Instalé **Docker Desktop para Windows** siguiendo la sección 1 del README (lo abrí y esperé a que indicara que estaba corriendo) y verifiqué la instalación:

```
docker --version
docker compose version
```

Salida obtenida:

```
Docker version 29.8.2, build 7fc2dff / Docker Compose version v5.6.0
```



Fase 2: Puesta en marcha

2.1 Clonar el repositorio

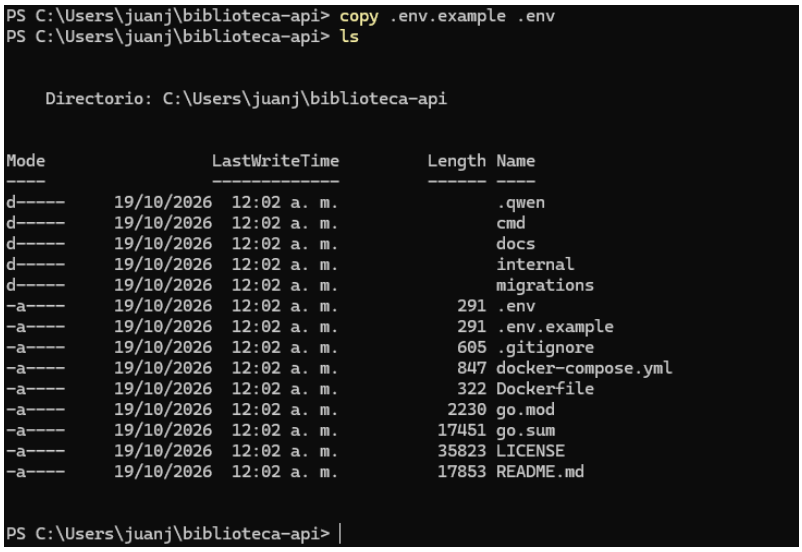
```
git clone https://github.com/carrenoutb/biblioteca-api.git
cd biblioteca-api
```

2.2 Piezas del proyecto (sección 2 del README)

- **`Dockerfile`:** receta para construir la imagen de la API. Es *multi-stage*: un paso compila el binario de Go y otro, más ligero, solo lo ejecuta.
- **`docker-compose.yml`:** define los dos servicios (`mysql` y `api`), los conecta en una red propia, expone puertos y lanza las migraciones.
- **`.env.example` → `.env`:** variables de configuración que no deben ir en el código. El `.env` no se sube al repo por contener datos sensibles.
- **`migrations/`:** scripts SQL que crean el esquema y cargan datos de ejemplo; MySQL los ejecuta la primera vez que arranca con la base vacía.

2.3 Crear el `.env`

```
copy .env.example .env
```



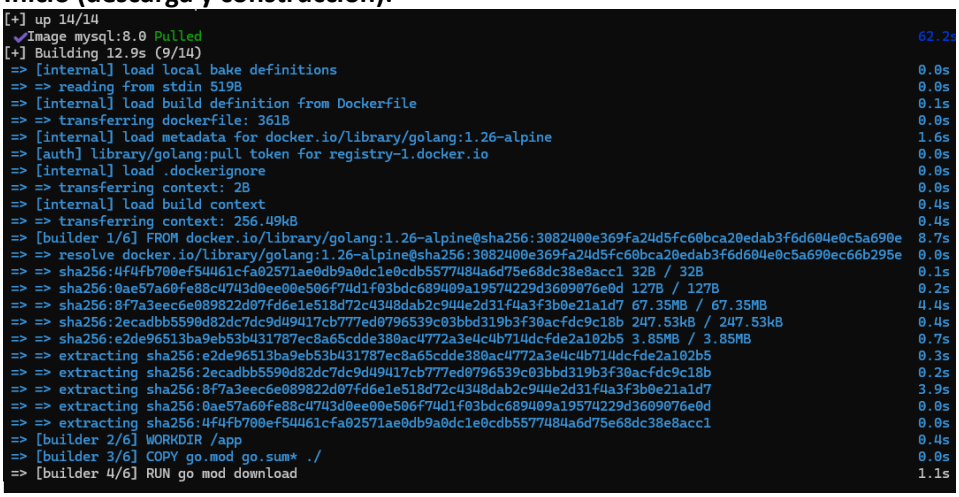
Variables explicadas:

Variable	Qué es
`DB_HOST`	Dónde vive la base de datos. Vale `mysql`, que es el nombre del servicio en la red de Docker, no una dirección de internet.
`DB_PORT`	Puerto de MySQL (`3306`).
`DB_USER`	Usuario de la base de datos (`biblioteca_user`).
`DB_PASSWORD`	Contraseña de ese usuario (`biblioteca_pass`).
`DB_NAME`	Nombre de la base de datos (`biblioteca`).
`JWT_SECRET`	Clave con la que se firman los tokens de autenticación.
`JWT_EXPIRATION_HOURS`	Horas de validez de un token (`24`).
`APP_PORT`	Puerto donde escucha la API (`8080`).

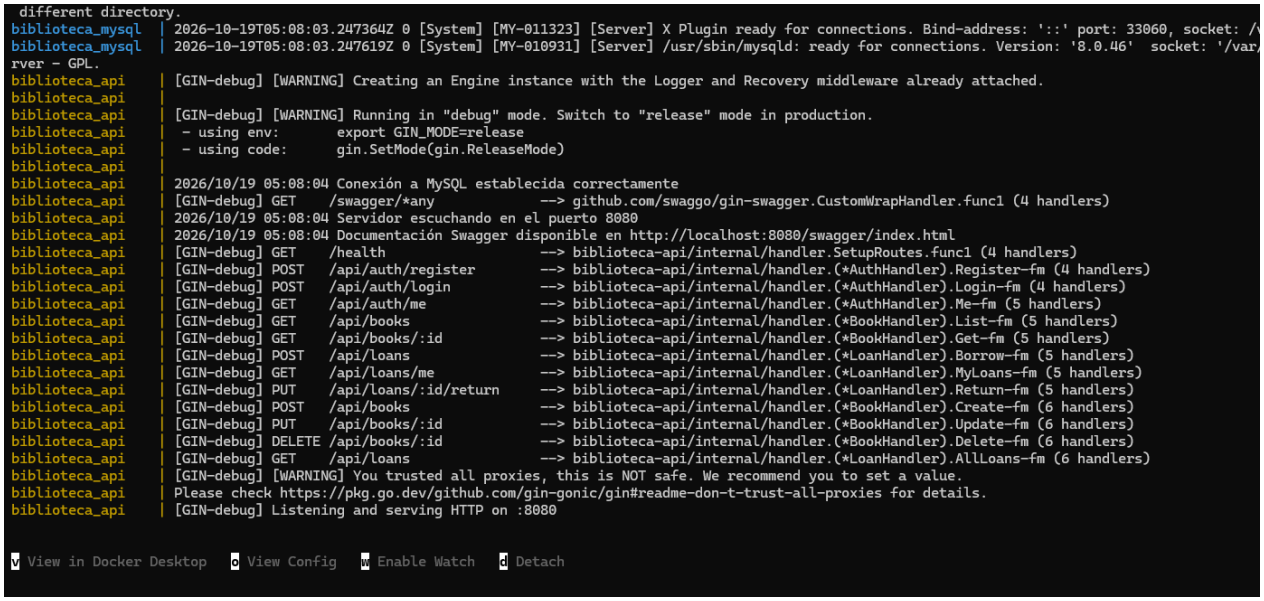
2.4 Levantar los servicios

```
docker compose up --build
```

Inicio (descarga y construcción):



Final (ambos servicios corriendo):



en este pasoDocker construyó la imagen de la API desde el Dockerfile, despues sedescargó la imagen de MySQL 8, arrancó MySQL y esperó a que esté sano (healthcheck), ejecutó las migraciones y por último arrancó la API en el puerto 8080.

Fase 3: Verificación

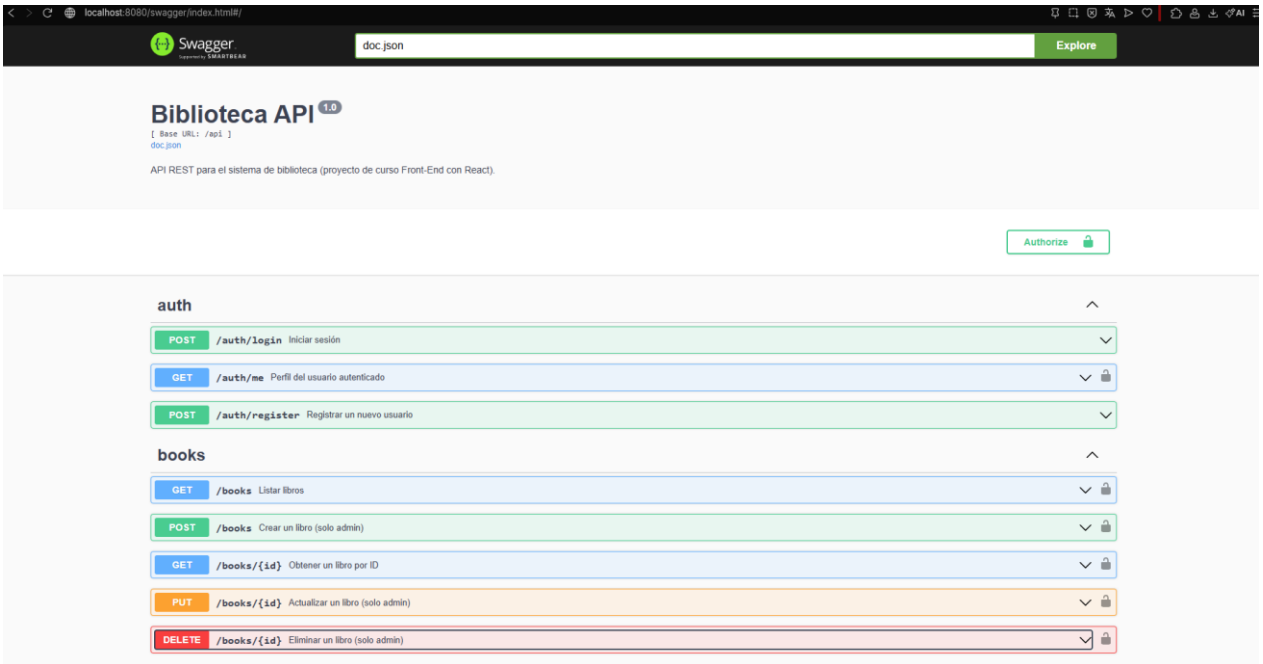
3.1 Estado de los servicios

```
docker compose ps
```

```
PS C:\Users\juanj\biblioteca-api> docker compose ps
WARN[0000] C:\Users\juanj\biblioteca-api\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME              IMAGE                 COMMAND                  SERVICE   CREATED         STATUS                   PORTS
biblioteca_api    biblioteca-api-api    "./biblioteca-api"       api       5 minutes ago   Up 4 minutes             0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
biblioteca_mysql  mysql:8.0              "docker-entrypoint.s…"   mysql     5 minutes ago   Up 4 minutes (healthy)   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
PS C:\Users\juanj\biblioteca-api>
```

3.2 Swagger

Abrí `http://localhost:8080/swagger/index.html` y cargó la documentación interactiva.



3.3 Flujo registrar → login → token → listar libros

Usé *REST Client de VS Code / Swagger UI* (indica cuál; en PowerShell `curl` es un alias de otro comando).

Paso	Petición	Resultado esperado
Health	`GET /health`	`{"status":"ok"}`
Registro	`POST /api/auth/register`	token + usuario con rol `student`
Login	`POST /api/auth/login`	token
Libros	`GET /api/books` con `Authorization: Bearer <token>`	lista de libros del seed

```
PS C:\Users\juanj\biblioteca-api> Invoke-RestMethod http://localhost:8080/health

status
------
ok
```

201

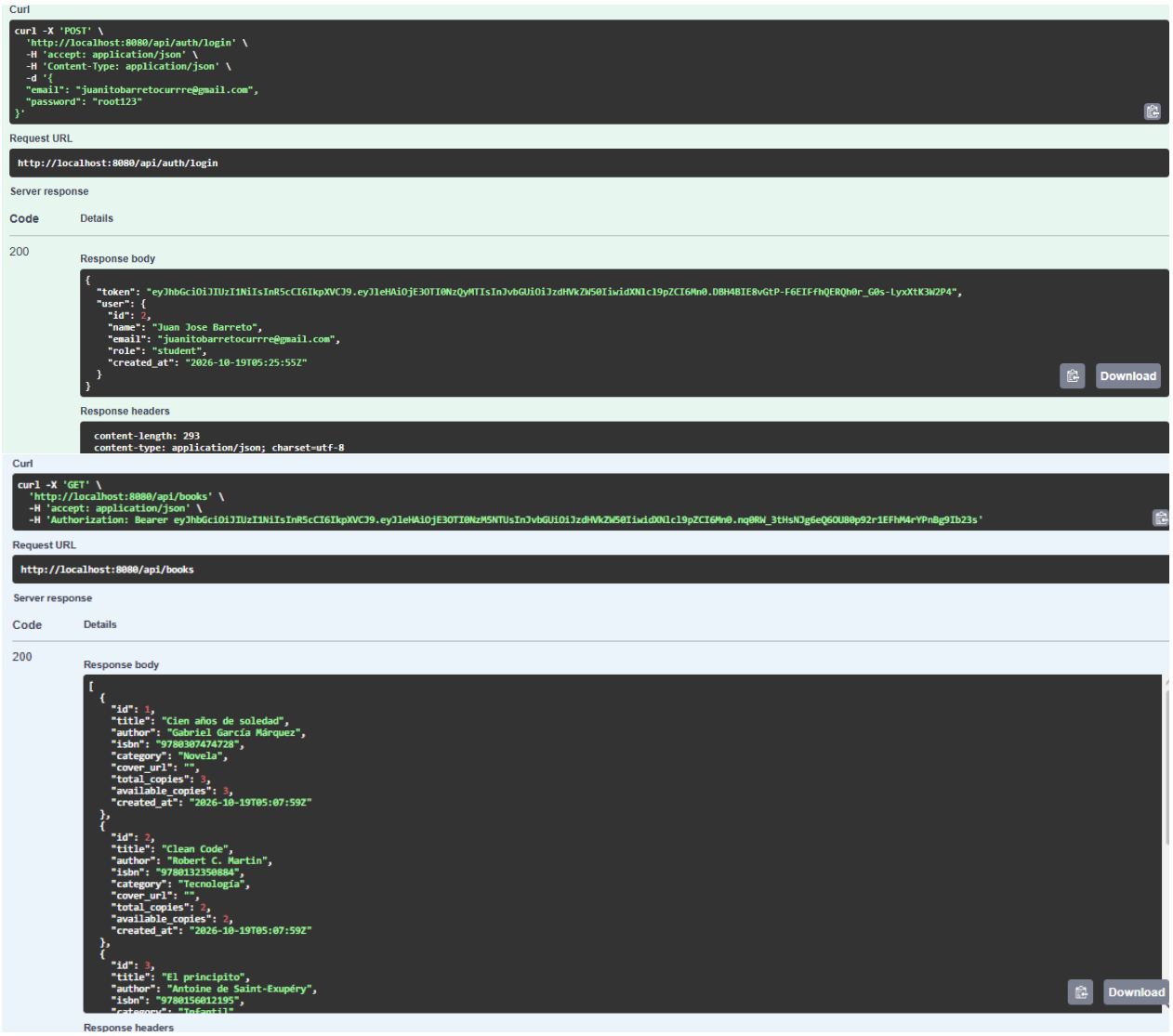
Response body


```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3OTI0NzM5NTUsInJvbGUiOiJzdHVkZW50IiwidXNlcl9pZCI6Mn0.nq0RW_3tHsNJg6eQ6OU80p92r1EFhM4rYPnBg9Ib23s",
  "user": {
    "id": 2,
    "name": "Juan Jose Barreto",
    "email": "juanitobarretocurrre@gmail.com",
    "role": "student",
    "created_at": "0001-01-01T00:00:00Z"
  }
}
```


Download

Response headers

```
content-length: 293
content-type: application/json; charset=utf-8
```



Problemas comunes (anexos del README)

1. `port is already allocated` **(3306 u 8080):** otra aplicación usa ese puerto. Solución: cerrarla o cambiar el mapeo en `docker-compose.yml` (por ejemplo `"8081:8080"`).
2. **404 al abrir** `http://localhost:8080/`**:** es normal, la API no tiene ruta raíz. Hay que usar `/health` o `/swagger/index.html`.

(no tuve ningun problema o error.)

Preguntas de comprensión

1. ¿Diferencia entre imagen y contenedor? La imagen es la plantilla de solo lectura; el contenedor es esa imagen en ejecución. En este proyecto, el `Dockerfile` produce la imagen de la API y `mysql:8` es una imagen descargada de Docker Hub. Al hacer `docker compose up`, de esas imágenes se crean los contenedores `api` y `mysql`. Con una misma imagen podría crear varios contenedores.

2. ¿Qué hace `docker compose` que no harías con `docker run` por separado? Con `docker run` tendría que crear la red, ejecutar cada contenedor con todos sus parámetros (puertos, variables, volúmenes) y controlar el orden a mano. Compose describe todo en un solo archivo y lo levanta con un comando: crea la red, conecta los servicios, aplica variables y volúmenes, y respeta dependencias como esperar a que MySQL esté sano antes de arrancar la API.

3. ¿Por qué `DB_HOST=mysql` y no `127.0.0.1`? Dentro del contenedor de la API, `127.0.0.1` se refiere al propio contenedor de la API, donde no hay ninguna base de datos. Compose crea una red en la que cada servicio se puede encontrar por su nombre, así que `mysql` se resuelve a la dirección del contenedor de MySQL.

4. ¿Qué borra `docker compose down -v` y cuándo usarlo? Detiene y elimina los contenedores y la red del proyecto, y además borra los volúmenes, es decir, los datos persistidos de MySQL. Lo usaría para volver a los datos de ejemplo originales: en el siguiente `up --build`, la base arranca vacía y se ejecutan de nuevo las migraciones y el seed. Hay que tener cuidado porque se pierden todos los datos.

5. ¿Qué pasa si cambio `DB_PASSWORD` en el `.env` pero no en `docker-compose.yml`, y al revés? Las credenciales que usa la API y las que usa MySQL deben coincidir. Si solo cambio una de las dos, la API intentará conectarse con una contraseña distinta de la configurada en MySQL y fallará con un error de acceso denegado (`Access denied`). Además, MySQL solo aplica las variables de creación de usuario cuando inicializa una base vacía; si el volumen ya existe, el cambio no se aplica hasta ejecutar `docker compose down -v`. *(Antes de entregar, revisa en tu `docker-compose.yml` cómo se definen esas variables, para confirmar esta respuesta con tu archivo real.)*

Conexión con la Parte 1

La cadena **Dockerfile → imagen → contenedor** se vio en la práctica con la API. El registro (Docker Hub) entró cuando se descargaron `golang` y `mysql`. Compose mostró cómo una arquitectura de varios servicios (API + base de datos) se levanta de forma reproducible en cualquier máquina.